BSc (Hons) Computing - Coventry University
Higher National Diploma in Software Engineering - NIBM

Data Warehousing and Business Intelligence

Niranga Dharmaratna
niranga@vl.nibm.lk

6.2 Data Modeling with Denormalized Concepts

What we are *to do*?

Focus on data modeling for warehouses, emphasizing ***denormalization*** to optimize query performance.

Discuss how denormalized structures differ from relational databases, making data access ***quicker and more efficient***.

Design data models using denormalized structures suited to data warehousing, enhancing data retrieval and analysis

6.2.1 Introduction to Data Modeling

- **Defines the structure of data**
- **Blueprint for data storage, retrieval, and usage**
- **Foundation for organizing data in warehousing**

Data Modeling

Data modeling is the process of creating a visual representation or a blueprint that defines the information collection and management systems of any organization.

This blueprint or data model helps different stakeholders, like data analysts, scientists, and engineers, to create a unified view of the organization's data.

The model outlines what data the business collects, the relationship between different datasets, and the methods that will be used to store and analyze the data. [AWS]

What is Data Modeling?

Blueprint for Organizing Data

- **Defines the structure and relationships within data**
- **Essential for efficient data storage and retrieval**

Purpose in Data Warehousing

- **Transforms raw data into a structured format**
- **Supports fast, accurate querying for analysis**

Types of Data Models

- **Conceptual Model: High-level view of data requirements**
- **Logical Model: Detailed structure without considering physical storage**
- **Physical Model: Specifies storage format and database design**

Activity: Data Modeling

URL:
https://aws.amazon.com/what-is/data-modeling/

Key Elements of Data Models

- Entities (e.g., Customer, Product)
- Attributes (e.g., Customer Name, Product ID)
- Relationships (e.g., Customer purchases Product)

Goals of Data Modeling in Warehousing

- **Optimize query performance**

  Structure data for faster access. Enable efficient reporting and analytics.

- **Simplify data retrieval**

  Organize large datasets for easy navigation. Streamline ETL (Extract, Transform, Load) processes.

- **Support decision-making processes**

  Provide reliable, structured data for business insights. Enhance data accuracy and relevance for decision-makers.

6.2.2 Normalization vs. Denormalization

Normalization

- Process of organizing data to reduce redundancy
- Breaks down data into smaller, related tables

- Ensures data accuracy and integrity
- Minimizes data duplication
- Common in Transactional Databases
- Supports daily operations with frequent updates
- Optimizes storage efficiency

Drawback in Data Warehousing

- Slows down complex queries due to multiple joins
- Less suitable for analytical, read-intensive tasks

Normalization

Continued...

Normalization typically involves dividing a large table into smaller, related tables and linking them using primary and foreign keys.

How It Works

- **Separation of Data: Each entity (customer, product, order) stored in its own table**
- **Relationships: Tables are linked by foreign keys (e.g., Customer ID in Orders Table)**

Normalization

Continued...

It follows a set of "normal forms," each with specific rules to organize the data

- **First Normal Form (1NF)**: Ensures each table column contains atomic (indivisible) values, and there are no repeating groups of columns.
- **Second Normal Form (2NF)**: Builds on 1NF by ensuring each non-primary key column is fully dependent on the primary key.
- **Third Normal Form (3NF)**: Further refines by removing transitive dependencies, where non-primary key columns depend on other non-primary key columns.

Normalization - Example

OrderID	CustomerID	CustomerName	CustomerCity	ProductID	ProductName	Price
1	101	Alice	New York	P01	Laptop	1200
2	102	Bob	Chicago	P02	Mouse	25
3	101	Alice	New York	P03	Keyboard	45

This table is prone to redundancy and update anomalies:

- **If a customer moves to another city, we'd need to update multiple rows.**
- **If we change the price of a product, it requires multiple updates.**

Normalization - Example

Continued...

- **1NF - Remove Repeating Groups**

  The table is already in 1NF since each column contains atomic values, and there are no repeating groups.

- **2NF - Remove Partial Dependencies**

  To reach 2NF, we separate data into two tables so that all non-key columns are fully dependent on the primary key.

  - **Orders Table (OrderID, CustomerID, ProductID)**
  - **Customers Table (CustomerID, CustomerName, CustomerCity)**

Normalization - Example

Continued...

- **3NF - Remove Transitive Dependencies**

  In 3NF, we eliminate transitive dependencies. In this case, ProductName and Price depend on ProductID, not OrderID.

  - **Orders Table (OrderID, CustomerID, ProductID)**
  - **Customers Table (CustomerID, CustomerName, CustomerCity)**
  - **Products Table (ProductID, ProductName, Price)**

Normalization - Example

Continued...

Final 3NF Structure

Now, each table has columns that depend only on their respective primary keys, achieving 3NF. This structure prevents redundant data and reduces anomalies, making the database easier to manage.

- **Orders Table links customers to products.**
- **Customers Table stores customer information.**
- **Products Table keeps product details.**

Denormalization

While normalization organizes data to eliminate redundancy and dependency issues, it can also lead to complex joins across multiple tables, which may slow down read operations, especially in large-scale applications.

Data denormalization is the process of combining tables or introducing redundancy back into a database structure to improve read performance.

Certain redundant data elements are reintroduced, or related data is consolidated into fewer tables. This reduces the number of joins necessary for queries, which can speed up data retrieval, making it more practical for read-heavy applications such as data warehousing or business intelligence.

Denormalization - Example

Using our Normalization-3NF example, suppose we want faster access to CustomerCity and ProductName for reporting. Instead of querying three separate tables, we could denormalize by combining data into a single Orders table.

OrderID	CustomerID	CustomerName	CustomerCity	ProductID	ProductName	Price
1	101	Alice	New York	P01	Laptop	1200
2	102	Bob	Chicago	P02	Mouse	25
3	101	Alice	New York	P03	Keyboard	45

This denormalized table avoids joins but introduces redundancy:

- **CustomerCity and CustomerName are repeated if a customer has multiple orders.**
- **If a product's Price changes, each instance needs to be updated.**

Denormalization

Continued...

When to Denormalize?

- **Read performance** is a priority, and fast data retrieval outweighs the storage or update cost.
- The database primarily handles **reporting** or **read-heavy workloads**.
- Data is less likely to change frequently, reducing the maintenance burden from redundancy.

The trade-off is that it increases storage and may complicate data consistency, so it's typically used carefully in scenarios where performance gains are necessary.

Key points - Normalization vs. Denormalization

N

- Organizes data to reduce redundancy
- Ensures data integrity
- Common in transactional databases
- Slower data retrieval
- Complex joins impact performance
- Not ideal for analytical workloads

D

- Combines data to reduce joins
- Allows for data redundancy
- Increases query efficiency
- Faster data retrieval for analytics
- Simplifies reporting
- Reduces dependency on complex joins
- Enhanced readability for analysts
- Better suited for BI and reporting

Challenges of Denormalization

Potential for Data Inconsistency
Redundant data may become outdated or incorrect. Requires strict data management to maintain accuracy.

Increased Data Redundancy
Duplicate data across tables to improve query speed. Leads to larger storage requirements.

Complexity in Data Updates
Changes to data must be applied in multiple locations. Increases workload on ETL (Extract, Transform, Load) processes.

Balancing Performance vs. Storage
Must weigh faster queries against increased storage and maintenance costs. Careful planning needed to avoid performance issues over time.

Data Integrity Risks
Higher chance of conflicting data in different tables. Needs additional validation to ensure consistency.

6.2.3 Denormalized Modeling - Key Concepts

Fact Tables

Store measurable data

- Store quantitative data, like sales amounts or order counts
- Represent key metrics for analysis (e.g., revenue, quantity)
- Contain foreign keys to link with dimension tables

Dimension Tables

Store descriptive data

- Store descriptive data, like product details or customer information
- Provide context to facts (e.g., time, location, product category)
- Typically smaller, more detailed tables

Discussion

Fact vs. Dimension

Facts:
What we measure (e.g., sales total).

Dimensions:
Describe "who, what, where, when" of facts

Let's have a deeper look...

Fact Tables - Characteristics

Primary Focus in Analytical Queries

Central to performance reporting and data insights. Drive business KPIs and trend analysis.

Typically Large in Size

Consolidate detailed transactional data. Can grow quickly with new data entries (e.g., daily sales records)

Contain Measurable Data

Store metrics and quantitative information for analysis. Examples: Sales amount, Order quantity, Profit margin

Foreign Keys Link to Dimension Tables

Central to performance reporting and data insights. Drive business KPIs and trend analysis.

Dimension Tables - Characteristics

Enable Data Filtering and Grouping

Allow analysis by categories, time periods, or locations. Make it easier to slice and dice data in queries.

Store Descriptive, Contextual Data

Provide the “who, what, where, when” around fact data. Describe facts with attributes.
Examples: Product Name, Customer Region)

Typically Smaller in Size

Contain reference data with fewer updates. Examples: Customer information, Product details, Time periods

Contain Primary Keys

Unique identifiers that link to foreign keys in fact tables. Essential for building relationships between facts and dimensions.

Discussion

Example of Fact and Dimension Table Structure

Scenario: Sales Analysis in a Retail Warehouse

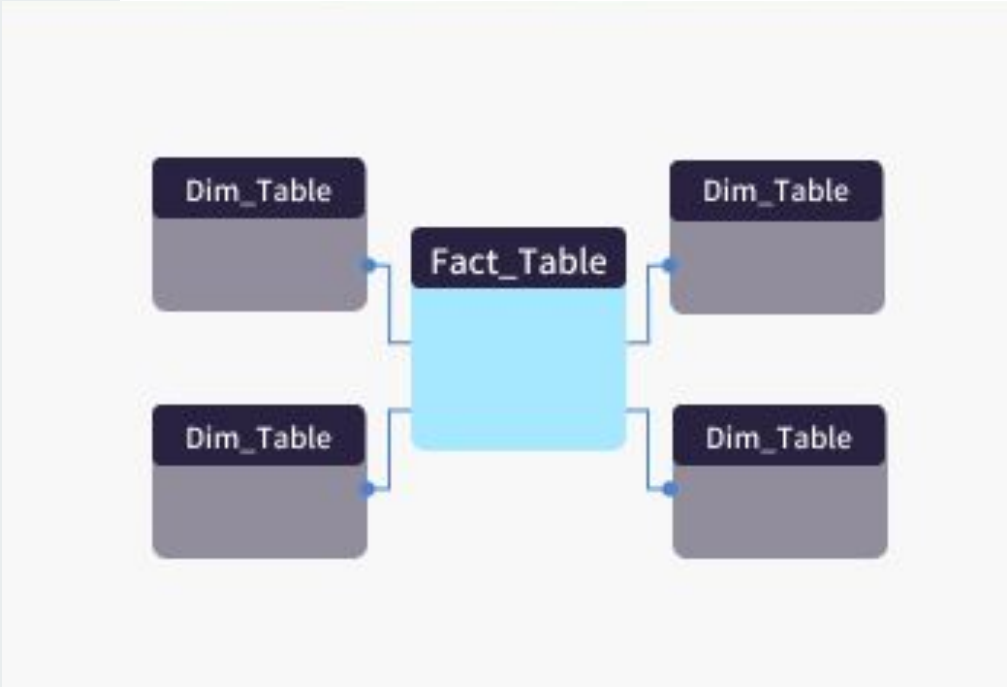
Fact Table (Sales)
Columns: *Sales ID, Date ID (FK), Product ID (FK), Store ID (FK), Customer ID (FK), Sales Amount, Quantity Sold*

Dimension Tables
Date: *Date ID, Day, Month, Year*
Product: *Product ID, Name, Category, Price*
Store: *Store ID, Location, Store Manager*
Customer: *Customer ID, Name, Age, Region*

Star Schema Structure

Simple structure with one fact table linked to multiple dimension tables

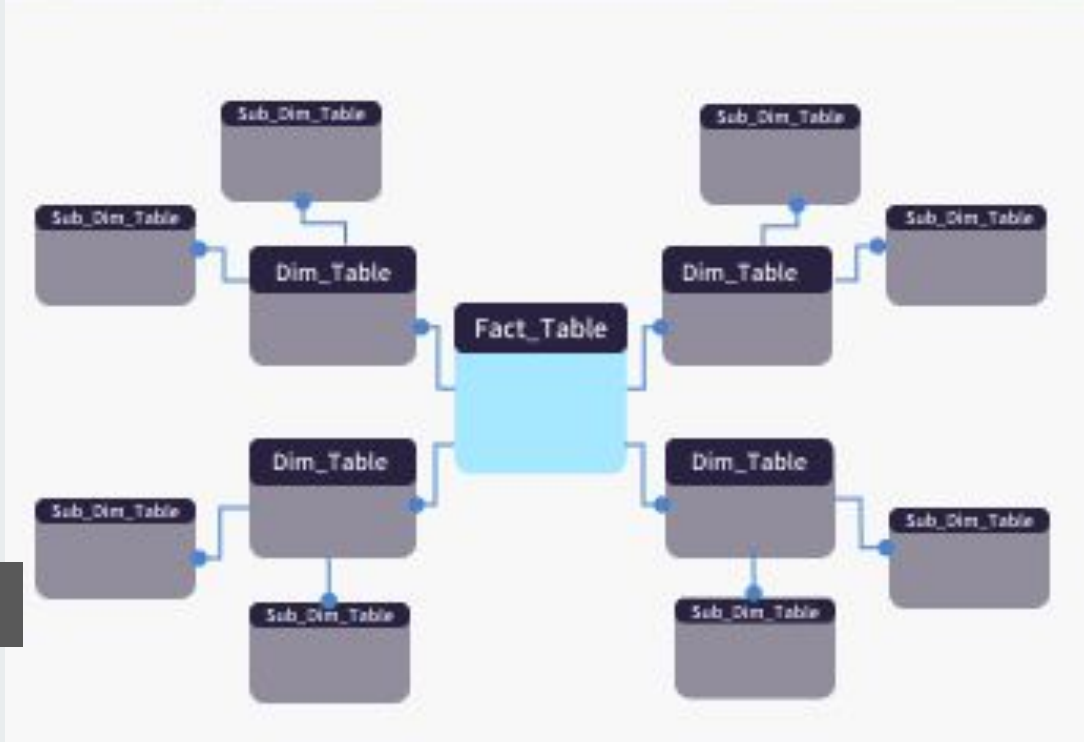
Easier to query, higher storage



Snowflake Schema Structure

Extended structure with normalized dimensions

More complex, but saves space



Granularity of Fact Tables

What?
The level of detail stored in a fact table. Determines how fine or coarse the data is.

Impact on Analysis

- **High Granularity: More detailed data, e.g., individual transactions**
- **Low Granularity: Aggregated data, e.g., daily or monthly sales totals**

Choosing Granularity Level

- **Must align with business needs and analytical goals**
- **Impacts data volume, storage, and query performance**

Trade-offs

- **Detailed Granularity: Enables detailed analysis but requires more storage**
- **Aggregated Granularity: Saves space and simplifies queries but limits drill-down**

Low Granularity Examples

Aggregated data

- **Monthly Financial Summary Table: Each row contains monthly totals for revenue and expenses**
- **Annual Sales Fact Table: Each row represents total sales by region for the entire year**

High Granularity Examples

More detailed data

- **Retail Sales Fact Table: Each row represents a single transaction**
- **Web Traffic Fact Table: Each row logs individual page views with timestamp and user ID**

Slowly Changing Dimensions (SCD)

Why They Matter?

Important for maintaining historical accuracy. Essential for accurate trend and pattern analysis over time.

What are SCDs?

Dimensions that change over time but at a slower pace compared to fact data
Used to track historical data changes in dimension tables

Common Examples

- Customer address updates
- Product price changes

Key Issues

- Balancing data accuracy with storage and query performance
- Determining how to handle changes without losing historical context

SCD Type 0: Ignore

Ignore changes!

- No changes are made; data remains static
- Pros: Simplest to manage with no additional maintenance
- Cons: No tracking of any changes, limits historical insights
- Example: Keeping product descriptions unchanged despite updates

SCD Type 1: Overwrite

Replace

- Replaces old data with new data
- Pros: Simple to implement
- Cons: Loses historical data
- Example: Updating a customer’s current address

SCD Type 2: Add New Row

Append Rows

- Adds a new row for each change, keeping history
- Pros: Retains full historical record
- Cons: Increases table size
- Example: Tracking changes in an employee’s job title over time

SCD Type 3: Add New Column

Append Columns

- Adds a column to store the old value
- Pros: Keeps limited historical data
- Cons: Only tracks one prior state
- Example: Tracking previous and current product category

Junk Dimensions

What are JDs?

A dimension that combines unrelated attributes that do not fit into other dimension tables. Helps reduce clutter and maintain efficient schema design.

Why They Matter?

- **Consolidates miscellaneous, low-cardinality attributes (e.g., flags, indicators, status codes)**
- **Simplifies data model by avoiding multiple small dimension tables**

- **Typically includes non-key, descriptive attributes**
- **Attributes grouped together are often Boolean or low-cardinality fields (e.g., “Yes/No” values)**

- **Reduces the number of dimension tables in a schema**
- **Helps maintain data organization and improve query performance**

Junk Dimensions - Examples

Attributes grouped together are often Boolean or low-cardinality fields (e.g., “Yes/No” values)

- **Combining attributes like IsDiscounted, PaymentType, and ReturnEligible into one junk dimension table**

Questions and Discussion

Disclaimer & Copyrights

The information presented in this presentation is intended for general informational purposes only. It is not meant to be comprehensive or to constitute professional advice. Any reliance you place on such information is strictly at your own risk. While I strive to keep the information accurate and up-to-date, I do make no representations or warranties of any kind, express or implied, about the completeness, accuracy, reliability, suitability, or availability concerning the content within this presentation.

This presentation may include references or links to third-party websites or content. I do not endorse the views expressed or the information provided on such external sites. I am not responsible for the content of any linked site or any link contained in a linked site.

Any action you take upon the information presented in this presentation is strictly at your own discretion. I will not be liable for any losses or damages in connection with the use of this information.

This disclaimer is subject to change without notice.

Last modified: 04 Nov 2024